

Integration Reliability for B2B SaaS

Four Game Design Iterations

How a Game Designer Would Build It

SyncTower · **IntegriPet** · **FailRun** · **PipeFlow**

Based on: Integration Reliability B2B SaaS Strategy Document

May 2026

1. What the Source Document Covers

The strategy document evaluates building a B2B SaaS product around integration reliability — preventing customer-visible API failures from causing churn, support ticket spikes, and engineering fire drills. The core market insight: 84% of B2B SaaS buyers say integrations are a key purchase requirement, 92% say integrated customers churn less, yet third-party APIs silently break through token expiry, schema drift, rate limit changes, and partial sync failures. Engineering teams treat integrations as “finished features” even though they’re living systems that degrade without warning.

The document recommends Option C (reliability and retention platform) as the strongest long-term play, with a phased approach: start with one integration family (e.g. Salesforce, HubSpot), ship alerting + replay/retry + root-cause summaries, then expand to multi-provider reliability workflows and finally a revenue protection reporting layer.

The ICP is a B2B SaaS company with 20–200 employees, 20+ customer-facing integrations, and a real support/churn cost when syncs break. Buyers are platform engineering leads, customer success heads, and revenue operations stakeholders.

The Design Problem for a Game Designer

The underlying challenge is attention and engagement. Integration health is invisible until it breaks. Nobody checks a monitoring dashboard at 9am out of curiosity — they check it at 2am because PagerDuty woke them up. The strategy doc itself identifies that “logs and metrics are not enough” and that the product must help teams resolve and recover, not just observe.

That’s a game design problem. You need to make an invisible, slowly-degrading system feel alive, urgent, and rewarding to maintain — the same challenge as keeping players engaged in a sim where nothing looks broken until everything collapses. The four iterations below each apply a different genre’s answer to that problem.

2. Four Iterations at a Glance

Dimension	SyncTower	IntegriPet	FailRun	PipeFlow
Genre	Tower Defence	Virtual Pet / Tamagotchi	Roguelike / Dungeon Crawler	Puzzle / Flow Optimisation
Vibe	Bloons TD meets PagerDuty	Tamagotchi meets Datadog	Slay the Spire meets incident response	Mini Metro meets data pipelines
Primary User	Platform engineer, on-call team	CS lead, product manager, team-wide	Senior engineer, incident responder	Integration architect, DevOps lead
Core Loop	Build defences → Waves hit → Patch → Upgrade	Feed → Check health → Heal → Grow	Enter incident → Pick tools → Diagnose → Resolve	Connect → Route → Balance → Score
Retention Hook	"Don't let the line break" tension	Emotional bond; guilt on neglect	Mastery; faster/better runs over time	Optimisation satisfaction; clean flow state

Monetisation Fit	Tiered by towers (monitored integrations) and wave complexity	Tiered by pet count (integration families) and care tools	Tiered by dungeon depth (incident complexity) and loadout	Tiered by pipe count and flow volume
------------------	---	---	---	--------------------------------------

3. Iteration 1: SyncTower

Tower Defence — "Bloons TD meets PagerDuty"

Core Concept

Every integration endpoint is a lane. Failure modes — token expiry, schema drift, rate limit spikes, partial syncs — are enemy waves that travel down these lanes toward your customers. Your job is to build, position, and upgrade defensive towers (monitoring rules, retry logic, alert routing, automated recovery scripts) that intercept failures before they reach customer accounts.

This isn't a metaphor painted over a dashboard. The spatial model — lanes, towers, waves, breach points — actually maps to how integration failures propagate: an upstream API change creates a failure event that travels through your sync pipeline and eventually hits customer data if nothing catches it.

Game Mechanics

The Map

Each integration (Salesforce, HubSpot, Slack, Zendesk) is a lane running left-to-right. Left side is the third-party API. Right side is your customer accounts. The lane visualises the actual data pipeline: auth → request → transform → write. Each stage is a tile where you can place towers.

Enemy Waves (Failure Modes)

Enemy Type	Real Failure Mode	Behaviour	Damage If It Reaches Customers
Token Ghost	Token expiry / refresh failure	Invisible until it hits the auth stage; silently stops all syncs on the lane	Complete sync halt; cascading support tickets
Schema Shifter	Field name / type changes	Appears normal initially then mutates mid-lane, breaking transform logic	Corrupted data, wrong field mappings, customer-visible errors
Rate Swarm	Rate limit / pagination changes	Arrives in large numbers; overwhelms throughput; causes partial syncs	Delayed or incomplete data; "where's my data?" tickets
Edge Creeper	Customer-specific custom field failures	Targets only one customer lane; hard to	Single-account failure that generates expensive escalation

		detect because most lanes stay green	
Silent Drifter	Gradual sync degradation	Moves very slowly; barely visible; accumulates damage over days/weeks	Stale data; customer loses trust in the product's accuracy

Towers (Defences)

- Sentinel Tower: Monitors a pipeline stage for anomalies. Detects enemies early. Upgrades improve detection sensitivity and speed.
- Retry Cannon: Automatically replays failed jobs. Handles transient failures. Upgrades add exponential backoff, selective retry, and batch replay.
- Schema Shield: Validates incoming data against expected schemas. Catches Schema Shifters at the transform stage. Upgrades add auto-mapping suggestions.
- Alert Beacon: Routes notifications to the right team. Doesn't stop enemies directly but accelerates human response. Upgrades add smart routing, deduplication, and escalation chains.
- Recovery Script (Special Ability): Deployable automated recovery. Cooldown-based. Resolves complex failures that towers alone can't handle — the equivalent of a special ability in TD games.

Scoring & Progression

Health Points: Each customer account has HP. Enemies that breach defences reduce account HP. Accounts at zero HP enter "at-risk" state (maps directly to churn risk from the strategy doc).

Wave Survival: Surviving a wave without any customer breach earns bonus points and unlocks tower upgrades.

Efficiency Rating: Graded on: incidents auto-resolved (no human intervention), mean time to detect, mean time to recover, support tickets prevented.

Why This Works as a Product

The tower defence model makes the invisible visible. Integration health isn't a dashboard metric — it's a spatial battlefield where you can see threats approaching, watch your defences intercept them, and feel the tension when something gets through. The strategy doc's key insight — that the product must help teams resolve, not just observe — is baked into the genre: towers act, they don't just report.

Maps to strategy doc: Phase 1 (narrow wedge) = one lane. Phase 2 (reliability workflow) = multiple lanes with cross-lane tower types. Phase 3 (revenue protection) = the scoring/HP system that ties incidents to account risk.

4. Iteration 2: IntegriPet

Virtual Pet / Tamagotchi — "Tamagotchi meets Datadog"

Core Concept

Each integration becomes a living creature — a small digital pet whose health, mood, and growth directly reflect the real-time state of that API connection. A healthy Salesforce sync is a happy, thriving pet. A degrading HubSpot connection is a sluggish, visibly unwell creature that needs attention. A dead integration is a pet that's collapsed and needs emergency care.

This sounds whimsical, but the psychology is precise. The reason integration health dashboards fail at engagement is that they're abstract. Humans don't form emotional bonds with uptime percentages. They do form bonds with creatures. The Tamagotchi model exploits a deep cognitive bias: people will check on something they feel responsible for, especially if neglect has visible consequences.

Game Mechanics

The Habitat (Dashboard Replacement)

Instead of a grid of status cards, the home screen is a habitat — a living ecosystem where your integration pets coexist. Healthy pets are active and animated. Sick pets show visible symptoms. The habitat's overall aesthetic reflects system health: lush and colourful when everything's running, grey and wilting when things degrade.

Pet Vital Signs (Mapped to Real Metrics)

Vital	What It Represents	Healthy State	Degraded State
Hunger	Data throughput / sync frequency	Well-fed; syncs arriving on schedule	Starving; syncs delayed or missing
Energy	Auth token health / API availability	Active, bouncing around	Exhausted, barely moving; token near expiry
Mood	Error rate / failure frequency	Happy, playful animations	Irritable, distressed; errors increasing
Growth	Cumulative reliability over weeks/months	Pet evolves to more impressive forms	Stunted; stuck at low evolution tier
Bond	Time since last human check-in / response speed	Pet greets you, trusts you	Pet is withdrawn, avoidant; you've been ignoring alerts

Care Actions (Mapped to Real Operations)

- **Feed:** Trigger a manual sync or backfill. Restores Hunger.
- **Heal:** Run a retry/replay on failed jobs. Fixes acute health drops.
- **Vaccinate:** Apply a preventive rule (schema validation, token refresh schedule). Prevents future illness.
- **Play:** Acknowledge an alert, review root-cause summary, close an incident. Restores Mood and Bond.
- **Evolve:** When a pet maintains healthy vitals for sustained periods, it evolves into a more advanced form. This is the long-term progression hook.

Notifications as Pet Distress Signals

Instead of "ALERT: Salesforce sync failure rate above threshold," the notification is your Salesforce pet looking unwell with a specific symptom. "Your Salesforce pet is losing energy

fast — auth token expires in 2 hours.” The emotional register drives faster response than a generic alert because you’re not dismissing a log entry, you’re neglecting a creature.

Team Dynamics

Shared habitats for teams. Everyone sees the same pets. When an engineer heals a pet, the whole team sees it recover. When nobody responds, the whole team sees the pet deteriorate. This creates gentle social pressure — the same mechanic that made Tamagotchi work in school playgrounds, now applied to on-call accountability.

Why This Works as a Product

The strategy doc identifies the core problem: integration health is invisible until it’s catastrophic. The Tamagotchi model makes degradation visible, emotional, and continuous. You don’t wait for a P1 incident — you notice your pet looks tired three days before the token expires. That’s the shift from reactive monitoring to proactive reliability that the strategy doc says is necessary but dashboards alone can’t achieve.

Maps to strategy doc: The “care actions” map directly to the doc’s requirement that the product must resolve, not just report. Feed = backfill. Heal = retry. Vaccinate = preventive rule. The evolution system maps to Phase 3’s revenue protection: a fully-evolved pet means months of sustained reliability, which is the data you show in executive reporting.

5. Iteration 3: FailRun

Roguelike / Dungeon Crawler — “Slay the Spire meets incident response”

Core Concept

Every integration incident is a dungeon. When a failure occurs, the responding engineer enters a procedurally-generated “run” through the incident’s root cause chain. Each room in the dungeon is a diagnostic step. Each encounter is a decision point: which tool to use, which path to investigate, which fix to attempt. The faster and more accurately you navigate the dungeon, the better your run score.

The roguelike model fits incident response surprisingly well because both domains share the same structure: enter an unknown situation, make sequential decisions with imperfect information, learn from each attempt, and get faster/better over time. The strategy doc’s emphasis on “root-cause summaries” and “recommended next actions” becomes the game’s dungeon map and item shop.

Game Mechanics

The Run (Incident Resolution Flow)

When an incident fires, the engineer opens the app and enters a run. The dungeon is generated from the actual incident data:

- Room 1 (Entry): Alert context — what broke, which customers affected, how many support tickets so far. The clock starts.
- Room 2 (Diagnosis Fork): Choose a diagnostic path — check auth tokens? Check schema? Check rate limits? Check upstream status page? Each choice costs time but reveals information.

- Room 3–N (Encounters): Each diagnostic step reveals findings. Some are dead ends (the token is fine). Some are the root cause. Some reveal secondary failures. The engineer makes decisions: retry now, escalate, apply a schema fix, refresh the token.
- Boss Room: The core fix. Apply the solution, verify recovery, confirm customer data integrity.
- Loot: Post-incident rewards — a root-cause summary auto-generated from the path taken, a preventive rule suggestion, an updated runbook.

The Loadout (Tooling)

Before entering a run, the engineer selects tools from their loadout:

- Quick Retry: Instant replay of failed jobs. Fast but doesn't address root cause.
- Schema Diff: Compares current vs expected schema. Reveals Schema Shifter encounters instantly.
- Token Inspector: Shows auth state, refresh history, expiry countdown.
- Log Lens: Deep log search with pattern matching. Slow but powerful.
- Customer Impact Scanner: Shows exactly which accounts are affected and their renewal dates.

Progression: The Codex

Every completed run adds an entry to the Codex — a growing knowledge base of resolved incidents. Over time, the Codex becomes the team's institutional memory. When a similar failure pattern appears, the Codex suggests the optimal path through the dungeon based on previous successful runs. This is the strategy doc's "failure pattern detection" turned into a progression system.

Scoring

Time Score: How fast was the run from alert to resolution? Faster = better.

Accuracy Score: How many dead-end rooms did you enter? Fewer wasted steps = better.

Impact Score: How many customer accounts were at risk? How many were protected by fast resolution?

Clean Run Bonus: Resolved without escalation or manual intervention beyond the initial responder.

Why This Works as a Product

Incident response is already a skill-based activity with clear success/failure states. The roguelike model adds structure to what's currently chaotic: instead of frantically grepping logs and pinging Slack channels, the engineer follows a guided decision tree that records every step. The "game" is getting faster and more accurate over time, which is exactly the operational goal.

Maps to strategy doc: Phase 1 (narrow wedge) = dungeons for one integration family. Phase 2 = the Codex becomes failure pattern detection and incident timelines. Phase 3 = Impact Score feeds directly into revenue protection reporting ("this run protected 14 accounts worth £340K ARR").

6. Iteration 4: PipeFlow

Puzzle / Flow Optimisation — "Mini Metro meets data pipelines"

Core Concept

The entire integration ecosystem is a flow network. Each API connection is a pipe. Data flows through pipes from third-party sources to customer accounts. The player's job is to design, balance, and maintain the network so data flows cleanly, efficiently, and without bottlenecks. When a pipe degrades, the network must be re-routed or repaired before downstream accounts are starved.

This is the purest systems-design iteration. It strips away narrative, creatures, and combat metaphors, and focuses entirely on the satisfying feeling of a well-running system — and the rising tension when flow starts backing up.

Game Mechanics

The Network (System View)

The home screen is a live, animated flow diagram. Left side: third-party API sources (Salesforce, HubSpot, Slack, etc.). Right side: customer account clusters. Between them: your pipe network. Data packets visibly flow through the pipes in real time. Healthy pipes flow smoothly. Degrading pipes show turbulence, colour shifts, narrowing diameter. Blocked pipes accumulate backed-up packets.

Flow Health (Real Metrics as Fluid Dynamics)

Visual Signal	What It Represents	Healthy State	Failing State
Flow speed	Sync frequency / throughput	Smooth, consistent particle flow	Slowing, stuttering, backing up
Pipe colour	Error rate	Green / blue	Yellow → orange → red
Pipe diameter	Rate limit headroom	Wide, flowing freely	Narrowing; approaching throttle
Particle shape	Data integrity	Uniform, clean shapes	Distorted; schema mismatch detected
Backpressure pool	Queue depth / retry backlog	No pooling	Visible accumulation at junction points

Player Actions

- Flush: Clear a backed-up pipe by triggering retry/replay. Immediate relief but temporary.
- Widen: Increase rate limit headroom or add parallel sync capacity. Costs resources.
- Reroute: When a source is down, redirect data flow through a cached or fallback path.
- Filter: Add schema validation at a junction point to catch corrupted packets before they propagate.
- Seal: Apply a permanent fix (token refresh automation, webhook reliability upgrade) that prevents future degradation on that pipe segment.

Scoring: Network Efficiency

- Flow Rate: Total data throughput across all pipes. Higher = better.
- Packet Loss: Data that didn't reach customer accounts. Lower = better.
- Backpressure Events: Number of times queues accumulated. Lower = better.
- Uptime Streaks: Consecutive days without a pipe going red. Longest streak = badge.
- Network Score: Composite grade displayed as a single number. The executive metric.

Why This Works as a Product

The flow network model makes system health intuitive at a glance. You don't read metrics — you see flow. A healthy system looks beautiful: clean, fast, balanced. A degrading system looks wrong before you even read a number. That perceptual immediacy is what dashboards with charts and tables can't achieve.

Maps to strategy doc: The Network Score is the executive reporting layer (Phase 3). Pipe-level actions are the operational workflow (Phase 2). Starting with one source's pipes is the narrow wedge (Phase 1). The visual language scales naturally from 1 integration to 50.

7. How Each Iteration Addresses the Strategy Doc's Key Requirements

Strategy Requirement	SyncTower	IntegriPet	FailRun	PipeFlow
Prevent customer-visible incidents	Towers intercept enemies before they reach customer HP	Pet symptoms appear before customer impact	Fast run completion limits blast radius	Flow degradation visible before pipe breaks
Resolve, not just observe	Towers actively block/retry; Recovery Scripts for complex fixes	Care actions = real operations (feed, heal, vaccinate)	Every room is a diagnostic action; Boss Room = fix	Flush, Widen, Reroute, Filter, Seal = real operations
Root-cause summaries	Post-wave report: which enemies got through, where, why	Pet health history = timeline of degradation events	Run path = auto-generated incident report	Pipe event log with visual replay
Account-level impact	Customer HP system; at-risk accounts visible on map	Each pet's bond to customer cluster visible	Impact Score shows affected accounts + ARR	Downstream customer nodes show starvation
Revenue protection reporting	Efficiency Rating + accounts-at-risk dashboard	Evolution milestones = sustained reliability evidence	Codex + cumulative run stats = ROI proof	Network Score trend = executive metric

Failure pattern detection	Wave patterns repeat; tower placement adapts	Recurring symptoms logged in pet health history	Codex identifies similar dungeon layouts across incidents	Pipe degradation patterns tracked across network
Phased roadmap (narrow → broad)	One lane → multi-lane → scoring system	One pet → habitat → evolution + reporting	One integration dungeon → Codex → impact scoring	One pipe → full network → Network Score

8. Strengths & Trade-offs

Iteration	Strengths	Trade-offs
SyncTower	Strongest spatial metaphor; makes threat propagation intuitive; towers = automation (product sells itself as the defence you're building); clear escalation from simple to complex; deeply satisfying to watch defences work	Spatial metaphor may feel forced for non-gamers; some engineering buyers may resist "game" framing; requires careful balance between game and real operational value
IntegriPet	Broadest team adoption (everyone checks on the pet, not just on-call); emotional engagement drives proactive monitoring; visual degradation catches problems before alerts fire; lowest cognitive barrier to entry; works for CS/product/engineering equally	Risk of not being taken seriously by enterprise buyers; "cute" aesthetic may conflict with B2B positioning; harder to map complex incidents to pet symptoms without oversimplifying
FailRun	Directly improves the real skill (incident response); Codex becomes genuine institutional knowledge; run scoring creates measurable improvement over time; cleanest mapping between game and operational reality; strongest for senior engineers	Only activates during incidents (not a daily engagement tool); doesn't address proactive prevention; requires enough incident volume to create progression; narrowest daily-use surface
PipeFlow	Most elegant visualisation; instantly communicates system health without reading numbers; scales naturally from 1 to 100 integrations; strongest executive appeal; closest to a real product you'd ship	Less gamified than the others; flow aesthetic may not sustain long-term engagement beyond novelty; player actions less differentiated than tower/pet/dungeon equivalents

9. Which One Would a Game Designer Actually Ship?

Depends on who you're selling to and what problem you lead with.

If the buyer is an engineering lead who wants operational tooling: PipeFlow. It's the most credible as a real product, the visualisation is genuinely better than dashboards, and the flow metaphor doesn't require the buyer to accept a "game" framing. It just looks and feels right.

If the buyer is cross-functional (CS + engineering + product) and the problem is team-wide awareness: IntegriPet. The emotional hook gets CS leads and product managers checking integration health voluntarily, which is something no monitoring dashboard has ever achieved.

If the buyer wants to improve incident response speed and quality: FailRun. It's the most skill-building iteration and produces a Codex that becomes the team's operational memory.

If you want the strongest portfolio piece that showcases game design thinking applied to B2B SaaS: SyncTower. The tower defence model is the most complete game design translation — enemies, towers, waves, progression, scoring — and it maps to the actual problem with surprising precision.

None of them are mutually exclusive. SyncTower's spatial view could be the monitoring layer. IntegriPet's emotional hooks could be the notification system. FailRun's dungeon structure could be the incident response workflow. PipeFlow's network view could be the executive dashboard. A complete product might use all four lenses for different audiences within the same company.